Java & Spring Boot
Memory Management Field Guide

JVM Memory Model • Garbage Collection Internals • OutOfMemoryError Troubleshooting • Memory Leak Diagnosis

Prepared as a practical reference for Java / Spring Boot microservice engineers, covering JVM internals, GC algorithms, and a step-by-step playbook for diagnosing and fixing OutOfMemoryError and memory leak issues in production.

Contents: JVM Memory Model • Garbage Collection & Collector Types • OutOfMemoryError (Symptoms, Root Causes, RCA, Tools, Fixes) • Memory Leaks (Symptoms, Root Causes, RCA, Tools, Fixes)

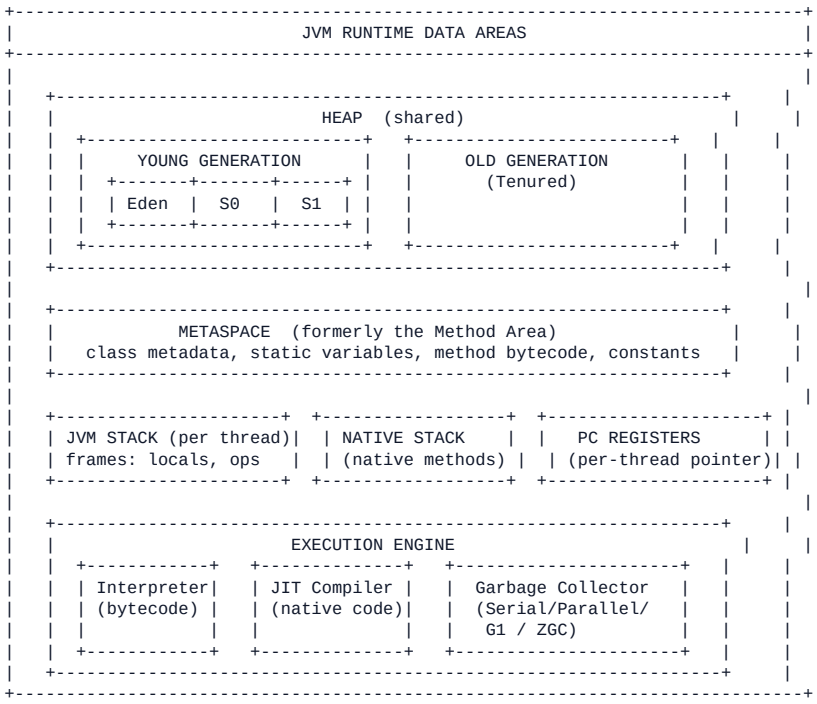
Table of Contents

1. JVM Memory Model
1.1 Memory Model Diagram
1.2 Component Functionality
2. Garbage Collection
2.1 Generational Hypothesis & Internal Working
2.2 Garbage Collector Types (Serial, Parallel, CMS, G1, ZGC)
2.3 Quick Reference Comparison
3. OutOfMemoryError — Java & Spring Boot Microservices
3.1 Symptoms
3.2 Root Causes
3.3 Root Cause Analysis (RCA) Process
3.4 Diagnostic Tools
3.5 Proposed Solutions & Fixes
3.6 Code-Level Fixes
3.7 Prevention Checklist
3.8 Summary RCA Checklist
4. Memory Leaks — Java & Spring Boot
4.1 Symptoms
4.2 Root Causes
4.3 RCA Step-by-Step
4.4 Diagnostic Tools
4.5 Code-Level Fixes
4.6 Immediate Mitigation
4.7 Prevention
4.8 Real-World Case Study

1. JVM Memory Model

1.1 Memory Model Diagram

The JVM divides its runtime memory into several distinct areas. The Heap is shared across all threads and is split into a Young Generation (Eden + two Survivor spaces) and an Old Generation (Tenured). Each thread gets its own Stack and Program Counter register, while class metadata lives in Metaspace, and the Execution Engine drives bytecode interpretation, JIT compilation, and garbage collection.



1.2 Component Functionality

Component	Functionality
Heap	Runtime area where every object and its instance fields are stored. Shared across all threads. Split logically into Young Generation (Eden + Survivor spaces S0/S1) and Old Generation to support generational garbage collection.
Metaspace (Class Area)	Stores class metadata, static variables, method bytecode and the constant pool. Since Java 8 it lives in native memory rather than the heap.
JVM Stack	One per thread. Holds stack frames — each frame stores local variables, the operand stack, and method call/return data.
Native Stack	Equivalent of the JVM stack, but used when a thread executes native (C/C++) code via JNI.
PC Register	Each thread has one program counter holding the address of the currently executing bytecode instruction. Undefined while executing native code.
Execution Engine	Runs the bytecode. Composed of the Interpreter (executes bytecode line by line), the JIT Compiler (compiles hot methods to native machine code), and the Garbage Collector (reclaims heap memory held by unreachable objects).

2. Garbage Collection

2.1 Generational Hypothesis & Internal Working

Garbage collection design rests on two empirical observations: most objects die young, and references from old objects to young objects are rare. This is why the heap is split into generations that are collected with different strategies.

Step-by-step lifecycle of an object under generational GC

- **Allocation** — new objects are placed in Eden, inside the Young Generation.
- **Minor GC** — when Eden fills up, a (usually fast) Stop-The-World pause runs. Reachable objects are copied out of Eden into a Survivor space using a copying algorithm; objects already in a Survivor space that are still alive move to the other Survivor space and their age counter increments.
- **Promotion** — once an object survives enough Minor GC cycles (or a Survivor space fills up), it is promoted into the Old Generation.
- **Major / Full GC** — when the Old Generation fills up, a more expensive collection runs, typically using mark-sweep-compact or a concurrent algorithm.
- **Finalization & de-allocation** — unreachable objects are identified, any finalizer is invoked, and their memory is returned to the heap for reuse.

2.2 Garbage Collector Types

Serial GC — -XX:+UseSerialGC

Single-threaded copying collector for Young Gen and single-threaded mark-sweep-compact for Old Gen. Every collection is a full Stop-The-World pause on one core.

- **Strengths:** zero synchronization overhead; smallest memory footprint.
- **Weaknesses:** cannot use multiple cores; pause time grows with heap size.
- **Best for:** single-core / embedded devices, heaps under ~100 MB, simple dev/test runs.

Parallel GC — -XX:+UseParallelGC (the “throughput” collector)

Uses multiple threads for both the Young Gen copying collection and the Old Gen mark-sweep-compact, but all phases remain Stop-The-World.

- **Strengths:** highest raw throughput; simple and well-tested; good CPU utilization.
- **Weaknesses:** pause times can grow unpredictably on large heaps; no pause-time target.
- **Best for:** batch jobs, ETL pipelines, overnight processing where throughput beats latency.

CMS — Concurrent Mark Sweep (deprecated in Java 9, removed in Java 14)

Runs most marking and sweeping concurrently with the application, pausing only briefly for initial-mark and remark. It never compacts the Old Generation.

- **Strengths:** very low pause times relative to Serial/Parallel.
- **Weaknesses:** Old Gen fragmentation eventually forces a long Stop-The-World compaction; higher CPU overhead; removed from modern JDKs.

- **Best for:** legacy Java 8 applications only — not recommended for new projects.

G1 GC — -XX:+UseG1GC (default since Java 9)

Divides the heap into many small regions (roughly 1–32 MB each) instead of two fixed generations. It concurrently marks live objects and then evacuates the regions containing the most garbage first — hence "Garbage First".

- **Phases:** Young-only evacuation, concurrent marking, mixed collections (Young + some Old regions), and Full GC as a rare fallback.
- **Strengths:** self-tuning; predictable pause-time goals via -XX:MaxGCPauseMillis; compacts memory during evacuation, avoiding fragmentation; scales well to multi-GB heaps.
- **Weaknesses:** somewhat lower throughput than Parallel GC; large ("humongous") allocations can trigger premature Full GCs.
- **Best for:** modern server-side applications, microservices, heaps above ~4 GB.

ZGC — -XX:+UseZGC (production-ready from Java 15)

A concurrent, compacting collector that performs almost all work — including compaction — without stopping the application, using colored pointers and load barriers to stay consistent while the app keeps running.

- **Strengths:** sub-millisecond pause times regardless of heap size, even at multi-terabyte scale; concurrent compaction avoids fragmentation.
- **Weaknesses:** higher CPU overhead from load barriers (roughly 5–10%); larger memory footprint for pointer metadata.
- **Best for:** latency-critical systems (trading, real-time inference), very large heaps.

2.3 Quick Reference Comparison

Feature	Serial	Parallel	CMS (legacy)	G1	ZGC
Concurrency	No	No	Partial (Old)	Partial (mixed)	Almost all
Compaction	Yes (STW)	Yes (STW)	No → fragments	Yes (evacuation)	Yes (concurrent)
Pause target	N/A	N/A	Low	Configurable	Sub-millisecond
Heap scale	<100 MB	<4 GB	<4 GB	>4 GB	Up to TBs
CPU overhead	Lowest	Low	Medium	Medium	Highest
Status	Opt-in	Old default	Removed (JDK14+)	Default (JDK9+)	Opt-in (JDK15+)

3. OutOfMemoryError — Java & Spring Boot Microservices

3.1 Symptoms

Category	What You Observe
Error logs	java.lang.OutOfMemoryError: Java heap space • GC overhead limit exceeded • Metaspace • Unable to create new native thread
Application behaviour	APIs return HTTP 500 or time out; the app hangs or slows sharply; Full GC runs continuously in the logs
Infrastructure / Kubernetes	Containers restart (exit code 137 / OOMKilled); memory alerts fire at ~95% of the pod limit; liveness/readiness probes fail
Business impact	Failed transactions, dropped orders, users unexpectedly logged out

3.2 Root Causes

Type	Typical Cause in Microservices
Heap OOM	Memory leaks (unclosed ThreadLocals, unbounded static caches); huge payloads (parsing large JSON/XML, fetching millions of DB rows without pagination); retaining entire result sets instead of streaming
Metaspace OOM	Excessive dynamic class loading / hot redeploys; heavy CGLIB proxy generation from Spring AOP; classloaders that are never released
GC overhead limit exceeded	The JVM spends almost all CPU time on GC but reclaims under ~2% of the heap — usually a heap that is too small for the live data set
Native / direct memory OOM	Off-heap leaks from ByteBuffer.allocateDirect(); Netty/Reactor buffers used by WebClient not released
Container memory limits	-Xmx set too close to the Kubernetes pod memory limit, leaving no headroom for Metaspace, stack, or off-heap use

3.3 Root Cause Analysis (RCA) Process

- **Step 1 — Capture the evidence:** enable permanent GC logging and a heap dump on OOM.
- **Step 2 — Analyze GC logs:** a heap that keeps growing after every Full GC points to a leak; a sudden spike points to a single oversized allocation.
- **Step 3 — Analyze the heap dump:** open the .hprof file in Eclipse MAT, inspect the Dominator Tree, and run the Leak Suspects report to find the top offending classes.
- **Step 4 — Check thread dumps:** use jstack to spot threads blocked on locks that are preventing objects from being released.
- **Step 5 — Correlate with metrics:** compare heap vs. non-heap usage and GC pause duration in Grafana/Prometheus over the incident window.

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/var/log/myapp/heapdump.hprof
-Xlog:gc*=info:file=/var/log/myapp/gc.log:time,uptime,level,tags
```

3.4 Diagnostic Tools

Tool	Purpose	Typical Usage
Eclipse MAT	Heap dump analysis	Dominator Tree + Leak Suspects report
JVisualVM	Live monitoring during development	Attach to local PID
JMC / Flight Recorder	Low-overhead production profiling	jcmd <pid> JFR.start
jstat	Real-time GC activity	jstat -gc <pid> 1000
jmap	Force an instant heap dump	jmap -dump:live,format=b,file=dump.hprof <pid>
jstack	Thread dumps for deadlocks/hangs	jstack <pid> > threads.txt
jcmd	General JVM diagnostics	jcmd <pid> GC.heap_info
Spring Boot Actuator	Cloud-friendly heap dump endpoint	GET /actuator/heapdump
GCeasy (online)	Visual GC log analysis	Upload the gc.log file
Grafana / Prometheus	Trend heap usage & GC pauses over time	Micrometer jvm.memory.* metrics

3.5 Proposed Solutions & Fixes

Immediate stabilization

Action	How
Increase heap / pod memory	Raise the Kubernetes memory limit and -Xmx (keep -Xmx below the pod limit)
Restart the affected pod	kubectl rollout restart deployment/<service> (temporary relief)
Scale out horizontally	Add replicas to spread the memory load
Disable problematic endpoints	If a specific heavy upload/report API is the trigger, temporarily block it at the Gateway/Ingress while a fix is prepared
Switch collector	Add -XX:+UseG1GC to cut Full GC frequency on larger heaps

Permanent code & configuration fixes

Problem Area	Fix
Unbounded caches	Use Caffeine/Spring Cache with maximumSize and expireAfterWrite
Large database queries	Replace full-table List<T> loads with Pageable or streaming queries
ThreadLocal leaks	Always call ThreadLocal.remove() in a finally block
RestTemplate / WebClient leaks	Use connection pooling and fully consume every response
Huge JSON payloads	Stream-parse with Jackson's JsonParser instead of loading a full String
Netty direct memory	Cap with -XX:MaxDirectMemorySize and release DataBuffers explicitly
Metaspace growth	Cap with -XX:MaxMetaspaceSize and avoid runtime dynamic class generation
Unclosed streams/connections	Use try-with-resources everywhere
Oversized HikariCP pool	Set spring.datasource.hikari.maximum-pool-size to a sane value (10-20)

3.6 Code-Level Fixes

Unbounded static cache → bounded Caffeine cache

```
// Before
private static final Map<String, String> cache = new HashMap<>(); // never evicts

// After
Cache<String, String> tokenCache = Caffeine.newBuilder()
        .expireAfterWrite(10, TimeUnit.MINUTES)
        .maximumSize(10_000)
        .build();
```

Full-table fetch → pagination

```
// Before
@GetMapping("/orders")
public List<Order> getAllOrders() {
    return orderRepository.findAll();      // millions of rows in memory
}

// After
@GetMapping("/orders")
public Page<Order> getOrders(Pageable pageable) {
    return orderRepository.findAll(pageable);
}
```

Unclosed stream → try-with-resources

```
// Before
InputStream is = file.getInputStream();
// ... never closed

// After
try (InputStream is = file.getInputStream()) {
    // process
}
```

3.7 Prevention Checklist

- Never set -Xmx equal to the container memory limit — leave 20-30% headroom for Metaspace, thread stacks, and off-heap buffers.
- Enable GC logging permanently in every environment, not just when something breaks.
- Add Prometheus alerts, e.g. warn when jvm_memory_used_bytes / jvm_memory_max_bytes stays above 0.85 for more than five minutes.
- Load-test with a profiler attached before shipping a change that touches caching, batching, or streaming code paths.

```
env:
- name: JAVA_OPTS
  value: "-Xmx512m -Xms512m -XX:+UseG1GC -XX:MaxMetaspaceSize=128m -XX:+HeapDumpOnOutOfMemoryError"
# Rule of thumb: Pod limit 1Gi -> -Xmx around 768m
```

3.8 Summary RCA Checklist — Spring Boot OOM

Step	Tool / Command	Output
1. Check logs	kubectl logs <pod> --previous	Look for OutOfMemoryError
2. Check GC logs	java -Xlog:gc*	If Full GC runs continuously → leak
3. Get heap dump	curl /actuator/heapdump or jmap	.hprof file
4. Analyze leak	Eclipse MAT (Leak Suspects)	Finds e.g. HashMap$Node or byte[] holding memory
5. Identify code	Match leak suspects to Spring beans / caches	e.g. @Cacheable without expiry
6. Apply fix	Code fix (pagination, eviction, try-with-resources)	Deploy new version
7. Validate	Load test + monitor Grafana	Memory settles at a stable baseline

Following this playbook lets you detect, diagnose, and resolve OOM issues in Spring Boot microservices quickly, while keeping the service highly available and stable.

4. Memory Leaks — Java & Spring Boot

A memory leak differs from a one-off OutOfMemoryError: it is gradual. Used heap creeps upward over hours or days until the application eventually crashes, rather than failing immediately after a single oversized allocation.

4.1 Symptoms

Category	Specific Symptoms
Heap metrics	jvm.memory.used trends steadily upward and never returns to baseline after a Full GC; Old Generation keeps growing toward -Xmx
GC logs	Full GC runs frequently but reclaims very little memory each time (e.g. 512M→510M)
Application behaviour	Response times creep up gradually; pods restart every few days (OOMKilled) rather than crashing right after deploy
Thread dumps	A growing number of WAITING/BLOCKED threads holding references to large object graphs

4.2 Root Causes

Root Cause	How It Happens in Spring Boot
Unbounded caches	@Cacheable without a TTL or size limit, or a static ConcurrentHashMap used as a cache that only grows
ThreadLocal misuse	A value is set on a pooled Tomcat thread (e.g. via a filter or RequestContextHolder) and never removed, so it outlives the request
Unregistered listeners	Event listeners or observers are registered in a global registry but never unregistered when the owning bean is destroyed
JDBC / resource leaks	ResultSet, Statement, Connection or InputStream objects are opened and never closed
Hibernate / JPA issues	@Transactional on a broad method that loads huge entity graphs, or EAGER fetch types pulling large object graphs into the session
Netty / WebFlux buffers	DataBuffer or ByteBuf instances are not released, causing an off-heap (direct memory) leak
ClassLoader leaks	Common with hot-reload tooling (e.g. devtools) or custom classloading where old classloaders are never freed
Static collections	A static List/Map in a utility class that objects are added to but never removed from

4.3 RCA Step-by-Step

- **Step 1 — Capture two heap dumps:** one shortly after startup (baseline) and one after several hours of load, via Actuator's /actuator/heapdump endpoint or jmap.
- **Step 2 — Compare in Eclipse MAT:** open the later dump's Histogram, use “Compare with another Heap Dump”, and sort by difference — the classes with the largest growth are the leak candidates.
- **Step 3 — Trace the GC root:** right-click the suspect class and run “Merge Shortest Paths to GC Roots” (excluding weak/soft references) to see exactly which thread or static field is holding the reference.

- **Step 4 — Confirm with GC logs:** a linear rise in Old Generation while Young Generation stays flat confirms objects are surviving and accumulating in Tenured space.

```
curl -X GET "http://my-app/actuator/heapdump" -o dump-A.hprof
# ... wait several hours under normal load ...
curl -X GET "http://my-app/actuator/heapdump" -o dump-B.hprof
```

4.4 Diagnostic Tools

Tool	Purpose	Leak-Specific Usage
Eclipse MAT	Heap dump analysis	Histogram comparison + Leak Suspects + GC-root tracing
JProfiler / YourKit	Live memory profiling	Watch live allocations and call stacks over time
Java Flight Recorder	Low-overhead profiling	jcmd <pid> JFR.start duration=60s
jcmd	On-demand heap dump	jcmd <pid> GC.heap_dump /path/heap.hprof
OQL (inside MAT)	Query a heap dump directly	SELECT * FROM java.util.HashMap WHERE size > 1000
Spring Boot Actuator	Expose heap dump safely	management.endpoints.web.exposure.include=heapdump
Grafana / Prometheus	Visualize the upward memory trend	Plot jvm.memory.used over days
Async Profiler	Low-overhead allocation profiling	Identify where most allocations originate

4.5 Code-Level Fixes

ThreadLocal leak in a servlet filter

```
// Before - missing cleanup
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {
    tenantHolder.set("tenant-1");
    chain.doFilter(req, res);
    // tenantHolder.remove() never called -> value stays on the pooled thread
}

// After
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {
    try {
        tenantHolder.set("tenant-1");
        chain.doFilter(req, res);
    } finally {
        tenantHolder.remove();
    }
}
```

Hibernate session holding a huge result set

```
// Before
@Transactional
@GetMapping("/orders")
public List<Order> getAll() {
    return repo.findAll();     // millions of rows stay attached to the session
}

// After - paginate
```

```
@GetMapping("/orders")
public Page<Order> getPaginated(Pageable pageable) {
    return repo.findAll(pageable);
}
```

Listener registered but never unregistered

```
// Before
@EventListener
public void onReady(ApplicationReadyEvent event) {
    Kafka.addListener(this::process);   // never unregistered
}

// After
@Component
public class MessageConsumer implements DisposableBean {
    private ListenerRegistration registration;

    @EventListener
    public void onReady(ApplicationReadyEvent event) {
        registration = Kafka.addListener(this::process);
    }

    @Override
    public void destroy() {
        if (registration != null) registration.unregister();
    }
}
```

Unclosed resources (file streams)

```
// Before
public String readFile(MultipartFile file) throws IOException {
    InputStream is = file.getInputStream();
    byte[] data = is.readAllBytes();
    // is never closed -> leaks the underlying buffer/file descriptor
    return new String(data);
}

// After
public String readFile(MultipartFile file) throws IOException {
    try (InputStream is = file.getInputStream()) {
        byte[] data = is.readAllBytes();
        return new String(data);
    } // auto-closed
}
```

Netty / WebFlux direct buffer leak

```
// Before
@GetMapping("/file")
public Mono<byte[]> download() {
    DataBuffer buffer = ...;               // allocates off-heap memory
    return Mono.just(buffer.asByteBuffer().array());
    // buffer.release() never called -> direct memory leak
}

// After
@GetMapping("/file")
public Mono<DataBuffer> download() {
    return DataBufferUtils.read(...)
        .doFinally(signal -> DataBufferUtils.release(buffer)); // released
}
```

4.6 Immediate Mitigation

Action	Command / Configuration
Restart the pod	kubectl rollout restart deployment/my-app
Increase heap / pod limit temporarily	Raise limits.memory and matching -Xmx
Force a Full GC to test if references are soft	jcmd <pid> GC.run
Capture a dump right before restart for later analysis	jmap -dump:live,format=b,file=crash.hprof <pid>

4.7 Prevention

- Set an eviction policy (maximumSize and expireAfterWrite) on every cache without exception.
- Keep @Transactional scoped to small, well-bounded service methods — never on a controller that can return unbounded data.
- Enable GC logging in every environment so an upward trend is visible long before it pages anyone.
- Use WeakHashMap or SoftReference for internal metadata caches where appropriate.
- Add a Prometheus alert on sustained heap growth, e.g. rate(jvm_memory_used_bytes[1h]) > 0.1 for 2h.
- Run a multi-hour load test with a profiler attached before every release that touches caching, filters, or listener registration — if the heap graph never plateaus, investigate before shipping.

4.8 Real-World Case Study (Simulated)

Stage	Detail
Symptom	The application restarted every 48 hours. Full GC ran every hour but reclaimed less than 5% of the Old Generation each time.
RCA using Eclipse MAT	Two heap dumps were compared. ConcurrentHashMap$Node showed 50,000 more entries in the later dump. Tracing the GC root produced the chain: Thread [http-nio-8080-exec-5] → ThreadLocalMap → CustomUserContext → List<Permissions>
Root cause	A security filter stored user permissions in a ThreadLocal but never called remove(), so each pooled thread accumulated stale permission lists over time.
Fix	Added a finally block calling threadLocal.remove() immediately after the filter chain completed.
Result	Heap usage stabilized at roughly 60%, Full GC frequency returned to normal, and pods ran stably for months without a restart.

End of document — a practical reference for diagnosing and resolving JVM memory issues in Java and Spring Boot microservice environments.